Laboratório 10

6TiSCH e RPL: Canais, DODAG e Trickle

Tecnologias de Comunicação para a Internet das Coisas | Aula 10

Objetivo e enquadramento

A Aula 10 apresenta o TSCH, com o seu salto de canal em tramas de intervalos, e o RPL, com o seu grafo orientado à raiz, a função objetivo e o Trickle Timer. Este laboratório reproduz esses mecanismos com um programa em Python, de modo que os números da aula possam ser calculados, alterados e comparados.

O trabalho não requer rádios, hardware nem máquina virtual, e corre em qualquer computador com Python 3. Destina-se a ser realizado durante a aula, ao ritmo de cada aluno, e **não constitui trabalho a entregar**.

Nota

O programa é um modelo didático. Calcula a frequência de uma célula TSCH com a fórmula da norma, escolhe pais com algoritmos de caminho mais curto e simula uma versão simplificada do Trickle Timer. Não substitui um simulador de rede.

Resultados de aprendizagem

- Calcular o canal real de uma célula TSCH em iterações sucessivas da trama.
- Construir um DODAG com uma função objetivo por saltos e outra por ETX, e comparar as escolhas de pai.
- Prever a reação da topologia à falha de uma ligação.
- Relacionar o intervalo do Trickle Timer com o volume de sinalização.

Requisitos

- Computador com Windows, macOS ou Linux e Python 3.9 ou superior.
- Duração estimada: entre 40 e 60 minutos.

Como utilizar este guião

Cada passo apresenta, por esta ordem, o objetivo, o enquadramento, o fundamento teórico, os comandos a executar e o resultado esperado. Recomenda-se ler o passo completo antes de executar o primeiro comando. As caixas têm sempre o mesmo significado:

Comando a executar

```
Comando a executar no terminal.
```

Resultado esperado (os valores podem variar)

```
Resultado que deve ser apresentado.
```

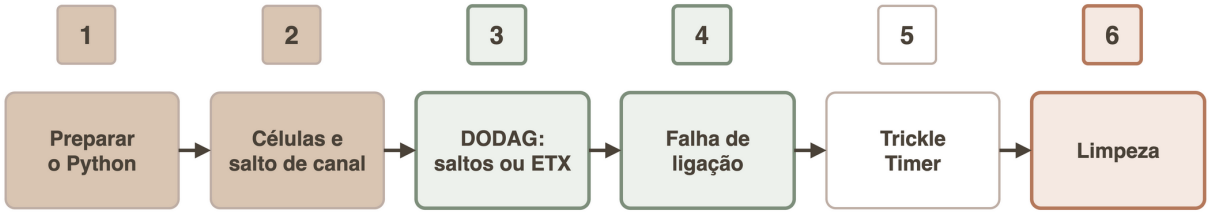
Analisar

Momento de pausa para interpretar o resultado obtido, isoladamente ou em discussão com os colegas.

Erro possível e resolução

Erro que pode ocorrer e respetiva resolução.

Percurso



Um único programa Python, sem bibliotecas externas: cada comando mostra um mecanismo da aula

Como ler a figura

Os seis passos leem-se da esquerda para a direita. O primeiro prepara o programa e o segundo calcula as frequências de uma célula TSCH. Os passos 3 e 4, a verde, constroem o DODAG e observam a reação a uma falha. O passo 5 mostra o Trickle Timer, e o passo 6 encerra o trabalho.

Passo 1 — Preparação do programa

Objetivo. Guardar o programa do laboratório e confirmar que o Python o executa.

Fundamento teórico

O programa usa apenas a biblioteca padrão do Python. Cada comando do programa corresponde a um passo do guião: `tsch`, `dodag`, `fail` e `trickle`. Sem argumento, executa os quatro.

Criar o ficheiro `rpl_lab.py` com o conteúdo seguinte. O ficheiro está também disponível na pasta `codigo` do material da aula.

Conteúdo do ficheiro `rpl_lab.py`

```
"""6TiSCH and RPL lab (Lecture 10): channel hopping, DODAG parent selection and
    Trickle timer.

Run:  python3 rpl_lab.py [tsch|dodag|fail|trickle]
No external libraries are needed.
"""
import heapq
import sys

# ---------------------------------------------------------------- TSCH
HOPPING = [16, 17, 23, 18, 26, 15, 25, 22, 19, 11, 12, 13, 24, 14, 20, 21]  # RFC
    8180 default
SLOTFRAME = 101


def channel(asn, channel_offset):
    """Actual IEEE 802.15.4 channel of a cell at a given Absolute Slot Number."""
    return HOPPING[(asn + channel_offset) % len(HOPPING)]


def tsch():
    slot, ch_off = 3, 1
    print(f"cell slotOffset={slot} channelOffset={ch_off}, slotframe of {SLOTFRAME
    } slots")
    for it in range(8):
        asn = it * SLOTFRAME + slot
        print(f"  iteration {it}: ASN={asn:4d} -> channel {channel(asn, ch_off)}")


# ---------------------------------------------------------------- DODAG
# link -> ETX (expected transmissions); the graph is undirected here for
```

```
    simplicity
LINKS = {
    ("R", "A"): 1.0, ("R", "B"): 2.4, ("R", "C"): 4.0,
    ("A", "C"): 1.8, ("B", "C"): 1.1, ("A", "D"): 1.1,
    ("C", "E"): 1.2, ("D", "E"): 2.0, ("D", "F"): 1.0, ("E", "F"): 1.3,
}


def neighbours(links):
    g = {}
    for (a, b), w in links.items():
        g.setdefault(a, {})[b] = w
        g.setdefault(b, {})[a] = w
    return g


def dodag(links, metric):
    """Dijkstra from the root R. metric 'hops' uses 1 per link, 'etx' uses the
    link ETX."""
    g = neighbours(links)
    cost = {"R": 0.0}
    parent = {}
    heap = [(0.0, "R")]
    while heap:
        c, u = heapq.heappop(heap)
        if c > cost.get(u, 1e9):
            continue
        for v, etx in g[u].items():
            nc = c + (1.0 if metric == "hops" else etx)
            if nc < cost.get(v, 1e9) - 1e-9:
                cost[v], parent[v] = nc, u
                heapq.heappush(heap, (nc, v))
    return parent, cost


def show(links, metric):
    parent, cost = dodag(links, metric)
    for n in sorted(parent):
        print(f"  {n}: parent {parent[n]}, rank/cost {cost[n]:.1f}")
    return parent


def dodag_cmd():
    print("Objective function by hop count:")
    p1 = show(LINKS, "hops")
    print("Objective function by ETX:")
    p2 = show(LINKS, "etx")
    diff = [n for n in sorted(p1) if p1[n] != p2[n]]
```

```
    print("nodes that choose a different parent:", ", ".join(diff) or "none")


def fail():
    print("Before the failure (ETX):")
    before = show(LINKS, "etx")
    broken = dict(LINKS)
    del broken[("A", "D")]
    print("After the failure of link A-D (ETX):")
    after = show(broken, "etx")
    print("parents that changed:", ", ".join(f"{n}: {before[n]}->{after[n]}" for n
     in sorted(before) if before[n] != after[n]) or "none")


# ------------------------------------------------------------------ Trickle
def trickle(total=300, imin=1.0, doublings=6, reset_at=None):
    """Instants at which a DIO is sent (simplified Trickle: one DIO at the middle
    of each interval)."""
    t, interval, sends = 0.0, imin, []
    imax = imin * (2 ** doublings)
    while t < total:
        end, mid = t + interval, t + interval / 2.0
        if reset_at is not None and t <= reset_at < end:
            if mid <= reset_at:
                sends.append(round(mid, 1))
            t, interval, reset_at = reset_at, imin, None   # inconsistency: back
    to Imin
            continue
        sends.append(round(mid, 1))
        t, interval = end, min(interval * 2, imax)
    return sends


def trickle_cmd():
    stable = trickle()
    print(f"stable network: {len(stable)} DIOs in 300 s; intervals grow to {1.0 *
    2 ** 6:.0f} s")
    print("first sends:", stable[:8])
    reset = trickle(reset_at=100)
    print(f"inconsistency at t=100 s: {len(reset)} DIOs in 300 s")
    after = [s for s in reset if s > 100][:6]
    print("sends right after the reset:", after)


CMDS = {"tsch": tsch, "dodag": dodag_cmd, "fail": fail, "trickle": trickle_cmd}

if __name__ == "__main__":
    which = sys.argv[1] if len(sys.argv) > 1 else None
```

```
    for name, fn in CMDS.items():
        if which in (None, name):
            print(f"== {name}")
            fn()
```

Comando a executar

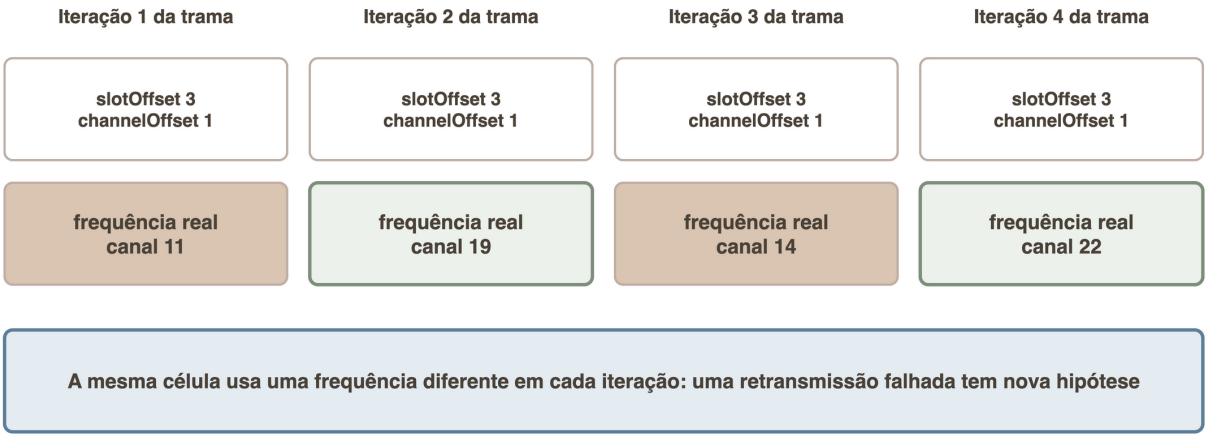
```
python3 rpl_lab.py tsch
```

Erro possível e resolução

python3: command not found em Windows: usar `python rpl_lab.py tsch`.

Passo 2 — Células TSCH e salto de canal

Objetivo. Calcular o canal real de uma célula em oito iterações da trama.



Como ler a figura

Cada coluna é uma iteração da trama. A caixa de cima indica a célula, que é sempre a mesma, e a de baixo a frequência real, que muda em cada iteração.

Fundamento teórico

A frequência de uma célula é dada por $F[(\text{ASN} + \text{channelOffset}) \bmod n]$, em que F é a sequência de saltos, n o seu comprimento e ASN o número absoluto do intervalo. Como o ASN cresce a cada iteração, a mesma célula cai numa frequência diferente. A sequência usada é a predefinida da RFC 8180, com 16 canais.

A analisar neste passo: a relação entre o ASN, o deslocamento de canal e a frequência.

Resultado esperado (os valores podem variar)

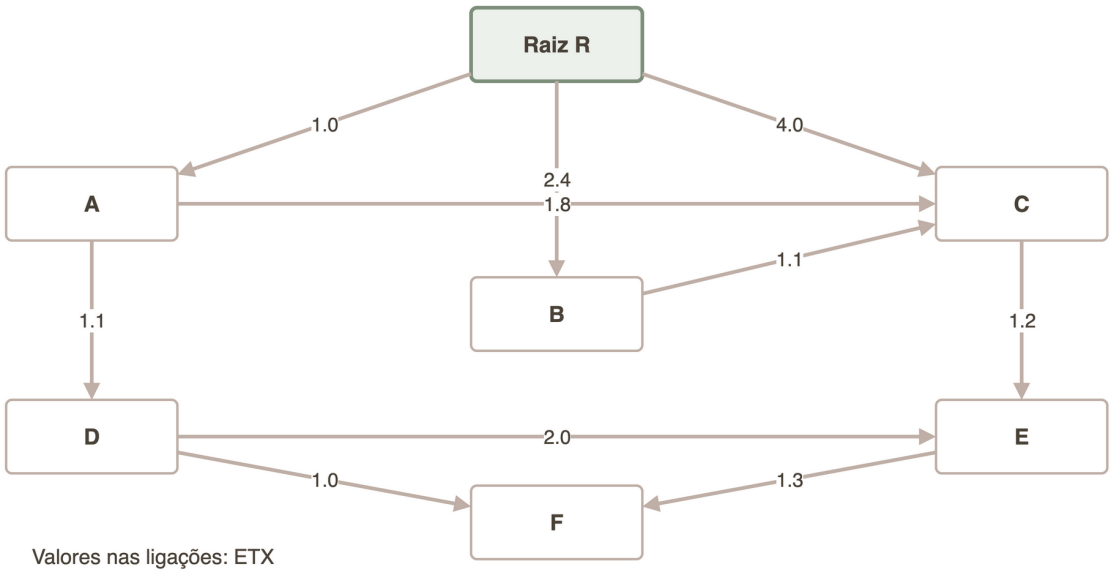
```
cell slotOffset=3 channelOffset=1, slotframe of 101 slots
  iteration 0: ASN=   3 -> channel 26
  iteration 1: ASN= 104 -> channel 11
  iteration 2: ASN= 205 -> channel 20
  iteration 3: ASN= 306 -> channel 18
  iteration 4: ASN= 407 -> channel 19
  iteration 5: ASN= 508 -> channel 14
  iteration 6: ASN= 609 -> channel 23
  iteration 7: ASN= 710 -> channel 22
```

Analisar

Altere `ch_off` para 2 e preveja os oito canais antes de executar. Duas células com o mesmo intervalo e deslocamentos de canal diferentes podem coincidir na mesma frequência numa iteração? Porquê?

Passo 3 — O DODAG: saltos ou ETX

Objetivo. Construir o grafo com duas funções objetivo e comparar o pai escolhido por cada nó.



Como ler a figura

O nó R é a raiz. Cada ligação tem o seu ETX, que estima o número de transmissões necessárias para a entregar com sucesso. A ligação direta entre R e C parece curta, mas tem um ETX de 4,0, ao passo que o caminho por A custa 2,8.

Fundamento teórico

A função objetivo por saltos soma 1 por ligação, e a função por ETX soma o ETX de cada ligação. O rank de um nó é o custo do melhor caminho até à raiz, e o pai preferencial é o vizinho por onde esse caminho passa. O programa usa o algoritmo de Dijkstra, que dá o caminho de custo mínimo, o mesmo resultado que o RPL obtém de forma distribuída.

A analisar neste passo: que nós mudam de pai e porque.

Comando a executar

```
python3 rpl_lab.py dodag
```

Resultado esperado (os valores podem variar)

```
Objective function by hop count:
  A: parent R, rank/cost 1.0
  B: parent R, rank/cost 1.0
  C: parent R, rank/cost 1.0
  D: parent A, rank/cost 2.0
  E: parent C, rank/cost 2.0
  F: parent D, rank/cost 3.0
Objective function by ETX:
  A: parent R, rank/cost 1.0
  B: parent R, rank/cost 2.4
  C: parent A, rank/cost 2.8
  D: parent A, rank/cost 2.1
  E: parent C, rank/cost 4.0
  F: parent D, rank/cost 3.1
nodes that choose a different parent: C
```

Analisar

O nó C escolhe pais diferentes nas duas funções. Qual é o custo real, em transmissões esperadas, do caminho que a função por saltos escolheu? E o nó E, que depende de C, tem o mesmo pai? O que acontece ao seu rank?

Passo 4 — Falha de uma ligação

Objetivo. Remover uma ligação e observar como a topologia se reorganiza.

Fundamento teórico

Quando uma ligação deixa de funcionar, os nós que a usavam como caminho de melhor custo recalculam o seu rank e podem trocar de pai. É o comportamento de reparação que o DODAG oferece sem intervenção do operador. O custo é, em regra, um rank maior e um caminho mais longo para os nós afetados.

Comando a executar

```
python3 rpl_lab.py fail
```

Resultado esperado (os valores podem variar)

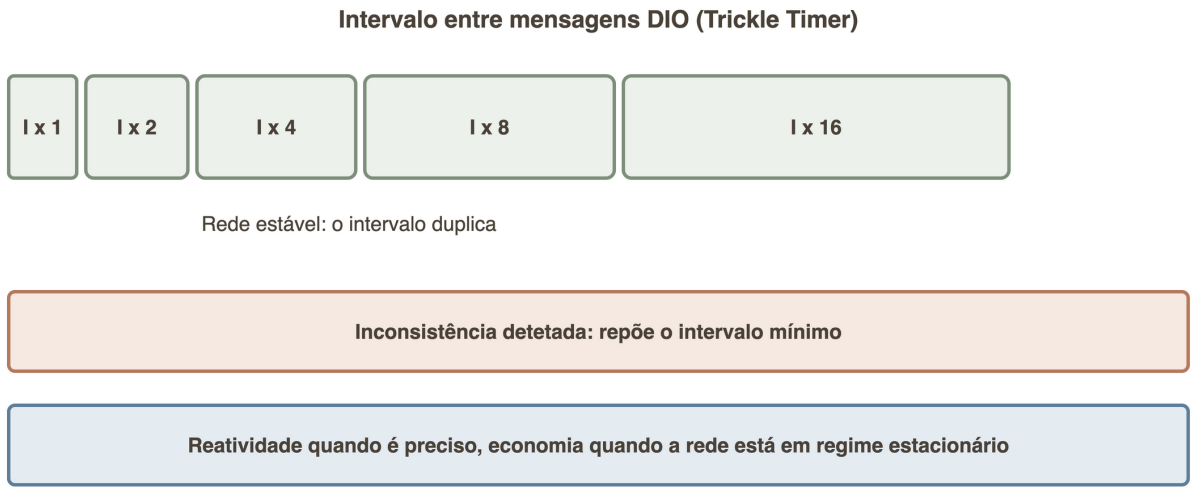
```
Before the failure (ETX):
  A: parent R, rank/cost 1.0
  B: parent R, rank/cost 2.4
  C: parent A, rank/cost 2.8
  D: parent A, rank/cost 2.1
  E: parent C, rank/cost 4.0
  F: parent D, rank/cost 3.1
After the failure of link A-D (ETX):
  A: parent R, rank/cost 1.0
  B: parent R, rank/cost 2.4
  C: parent A, rank/cost 2.8
  D: parent E, rank/cost 6.0
  E: parent C, rank/cost 4.0
  F: parent E, rank/cost 5.3
parents that changed: D: A->E, F: D->E
```

Analisar

Quais os nós que mudaram de pai e quais mantiveram o pai mesmo com um rank pior? Altere o programa para remover a ligação C–E e preveja o resultado.

Passo 5 — O Trickle Timer

Objetivo. Observar como o intervalo entre DIO cresce numa rede estável e é reposto quando se deteta uma inconsistência.



Como ler a figura

Os blocos verdes representam intervalos cada vez maiores, que duplicam enquanto a rede se mantém estável. A caixa vermelha indica o que acontece quando é detetada uma inconsistência.

Fundamento teórico

O Trickle Timer duplica o intervalo entre emissões até um máximo, enquanto os vizinhos anunciam informação coerente. Ao detetar uma inconsistência, o intervalo regressa ao mínimo, e a nova informação propaga-se em segundos. O modelo do programa emite um DIO a meio de cada intervalo, que é uma simplificação do mecanismo real, no qual o instante é aleatório e há supressão de mensagens redundantes.

A analisar neste passo: o número de mensagens emitidas em 300 segundos com e sem inconsistência.

Comando a executar

```
python3 rpl_lab.py trickle
```

Resultado esperado (os valores podem variar)

```
stable network: 10 DIOs in 300 s; intervals grow to 64 s
first sends: [0.5, 2.0, 5.0, 11.0, 23.0, 47.0, 95.0, 159.0]
inconsistency at t=100 s: 16 DIOs in 300 s
sends right after the reset: [100.5, 102.0, 105.0, 111.0, 123.0, 147.0]
```

Analisar

Quantos DIO adicionais custou a inconsistência? Como variaria este número se o Imin fosse de 4 s? Que vantagem tem uma rede estável quase não emitir sinalização num dispositivo a bateria?

Passo 6 — Limpeza

Não ficou nada instalado: basta apagar o ficheiro `rpl_lab.py`.

Questões de consolidação

1. Porque é que a mesma célula TSCH cai em frequências diferentes em iterações sucessivas?
2. Em que situação a função objetivo por saltos escolhe um caminho pior do que a por ETX?
3. Que nós têm de recalcular o rank quando uma ligação falha?
4. O que muda no consumo de energia quando o Trickle Timer chega ao intervalo máximo?

Síntese

- O **salto de canal** faz uma célula usar frequências diferentes em cada iteração, o que estabiliza as ligações.
- O **DODAG** escolhe pais pelo critério da função objetivo, e a função por ETX protege contra ligações más.
- A **reparação** é automática: os nós afetados recalculam o rank e trocam de pai.
- O **Trickle Timer** reduz a sinalização em regime estável e reage depressa a alterações.